

Assignment's Software Engineer

School Backoffice System — Attendance & Credit Module

This is a take-home assignment with a 6-hour time budget. The flexibility allows you to fit this around your schedule — but we ask that you limit your actual working time to 6 hours and track it honestly. We are not looking for a complete system. We are looking at how you think, what you prioritise, and how much you can deliver when time is the constraint.

🕒 6-hour time budget · 🤖 AI tools encouraged · ⚙️ Stack: your choice with justification

Tech stack

In your README, include a section called Stack decisions listing each major tool and one or two sentences on why you chose it over the alternatives. This is part of the evaluation.

System context

A centralised backoffice platform for a children's coding school with multiple branches. HQ and branch staff manage students, classes, bookings, and credits through a shared system with role-based access. Your assignment covers the most business-critical module: how the system tracks attendance and deducts student credits correctly across all scenarios.

Core business rules — the four attendance states

- Booked — Seat reserved Credit deduction: 0 Seat impact: reduces available seats by 1 in the class
 - Attend — Attended class Credit deduction: -1 Seat impact: none — uses the seat reserved at booking
 - Skip — Approved leave Credit deduction: 0 Seat impact: seat is consumed operationally Entitlement: system must compensate the student — via makeup class, seat credit, or extended course expiry — so they receive the full number of sessions in their package
 - Absent — No-show Credit deduction: -1 Seat impact: seat is forfeited — no compensation granted
-

Assignment scope

- Part 1 — System & database design
- Part 2 — Backend implementation
- Part 3 — Frontend interface
UI does not need to be pixel-perfect — clarity and correctness matter more than aesthetics.
- Part 4 — Quality & engineering practices
Demonstrate that you write production-aware code.

Bonus: Security considerations (auth, input sanitisation, rate limiting) are not required but will be noted positively

AI usage — expectations

AI tools are fully permitted and encouraged. We are not testing whether you can code without AI — we are testing whether you can use AI intelligently as a senior collaborator rather than a copy-paste machine.

- Include an AI usage log: for each significant AI interaction, note what you asked, what it produced, and what you changed or rejected — and why
 - Be prepared to explain every line of code in your presentation. If you cannot explain it, do not submit it
 - We value candidates who direct AI to solve the hard parts (schema design, transaction logic, edge cases) rather than only using it for boilerplate
-

Deliverables

1. GitHub repository (public or shared) Clean commit history. README with setup instructions, Stack decisions section, architecture summary, what is complete, and what you would do next with more time.
 2. Design document ERD, schema, architecture diagram, stack justification, and written rationale for your key decisions. Can live in the README or as a separate file.
 3. AI usage log (PDF, Markdown, or Google Doc) Key prompts, what you accepted, what you corrected, and your reasoning.
 4. Live presentation — Tuesday 19 May 2026 15–20 minutes. Walk us through your design decisions, demo the working system, justify your stack choices, and explain your AI collaboration approach. We will ask questions.
-

Evaluation rubric

	Area	Weight
1	System & database design — schema correctness, normalisation, indexing, constraints, ability to justify every decision	25%
2	Business logic accuracy — all four states handled correctly, atomic transactions, edge cases considered	25%
3	Code quality & engineering practices — readable code, tests, clean Git, API docs, performance awareness, stack justification	20%
4	AI collaboration & awareness — directed AI on hard problems, can explain all code, usage log shows critical thinking	20%
5	Communication & presentation — clear design doc, surfaces and articulates assumptions made, confident Q&A, explains trade-offs and next steps	10%

Bonus: Security considerations will be noted positively but are not required.

One last note ...

We know you're giving up personal time for this, likely over the weekend. We don't take that lightly. We've tried to make the problem interesting enough to be worth it — and we promise to give you honest, useful feedback regardless of the outcome.

We have designed this to be genuinely interesting to work on. The business logic has real edge cases, the constraints are real, and the problems you will encounter are the same ones our team deals with. We are not looking for perfection — we are looking for how you think under pressure, how you communicate what you built and what you didn't, and whether we'd enjoy building with you every day.

Good luck :)